11. Analyzer Class (Dynamic Memory)

Program:

cpp

```
#include <iostream>
using namespace std;

class Analyzer {
   int *arr, n;
public:
   Analyzer(int size) { n=size; arr=new int[n]; }
   void input() { for(int i=0;i<n;i++) cin>>arr[i]; }
   void analyze() {
      int max=arr[0], min=arr[0], sum=0;
      for(int i=0;i<n;i++) {
         if(arr[i]>max) max=arr[i];
         if(arr[i]<min) min=arr[i];
         sum+=arr[i];
      }
      cout<<"Max="<<max<<" Min="<<min<<" Avg="<<(double)sum/n<<endl;
   }
   ~Analyzer() { delete[] arr; }
};

int main() {
   int size; cin>>size;
   Analyzer a(size);
```

```
    a.input();
    a.analyze();
    return 0;
}
```

Logic: Dynamic array with new. Finds max, min, average. Destructor frees memory. **Example:** Input: 5 → 10 20 30 40 50 → Output: Max=50 Min=10 Avg=30

14. Decimal to Binary (Class)

Program:

cpp

```
#include <iostream>
using namespace std;

class DecimalToBinary {
    int num;
public:
    void input() { cin >> num; }
    void convert() {
        int bin[32], i = 0, n = num;
        while (n > 0) { bin[i++] = n % 2; n /= 2; }
        for (int j = i - 1; j >= 0; j--) cout << bin[j];
    }
};

int main() {
    DecimalToBinary d;
    cout << "Enter decimal number: ";
```

```
    d.input();
    cout << "Binary: ";
    d.convert();
    return 0;
}
```

Logic: Divide by 2 repeatedly, store remainders, print in reverse. **Example:** Input: 10 → Output: Binary: 1010

17. Student Record System (Map)

Program:

cpp

```
#include <iostream>
#include <map>
#include <fstream>
using namespace std;

class StudentRecord {
    map<int, string> records;
public:
    void addRecord() {
        int roll; string name;
        cin >> roll >> name;
        records[roll] = name;
    }
    void displayRecords() {
        for (auto &p : records) cout << p.first << " " << p.second << endl;
    }
```

```
    void saveToFile() {
        ofstream fout("students.txt");
        for (auto &p : records) fout << p.first << " " << p.second << endl;
        fout.close();
    }
    void menu() {
        int choice;
        do {
            cout << "\n1.Add 2.Display 3.Save 4.Exit\nChoice: ";
            cin >> choice;
            if (choice == 1) addRecord();
            else if (choice == 2) displayRecords();
            else if (choice == 3) saveToFile();
        } while (choice != 4);
    }
};

int main() {
    StudentRecord s;
    s.menu();
    return 0;
}
```

Logic: Uses map<int,string> for roll→name. Menu-driven add/display/save. **Example:** Input: 101 Komal, 102 KG → Output:

Code

101 Komal

102 KG

18. Shopping Cart System (Vector + Exception)

Program:

cpp

```
#include <iostream>
#include <vector>
using namespace std;

class Item {
public:
   int itemId;
   string itemName;
   double price;
};

class ShoppingCart {
   vector<Item> cart;
public:
   void addItem() {
      Item it;
      cin >> it.itemId >> it.itemName >> it.price;
      if (it.price < 0) throw invalid_argument("Invalid price!");
      cart.push_back(it);
   }
   void displayItems() {
      for (auto &it : cart)
```

```
        cout << it.itemId << " " << it.itemName << " " << it.price << endl;
    }
    void totalBill() {
        double sum = 0;
        for (auto &it : cart) sum += it.price;
        cout << "Total Bill = " << sum << endl;
    }
    void menu() {
        int choice;
        do {
            cout << "\n1.Add 2.Display 3.Total 4.Exit\nChoice: ";
            cin >> choice;
            try {
                if (choice == 1) addItem();
                else if (choice == 2) displayItems();
                else if (choice == 3) totalBill();
            } catch (exception &e) { cout << e.what() << endl; }
        } while (choice != 4);
    }
};

int main() {
    ShoppingCart s;
    s.menu();
    return 0;
}
```

Logic: Uses vector for items. Exception if price < 0. Menu-driven. **Example:** Input: 101 Pen 20, 102 Book 50 → Output:

Code

101 Pen 20

102 Book 50

Total Bill = 70

19. String Functions Using Pointers

Program:

cpp

```
#include <iostream>

using namespace std;


int strLength(char *s) {

   int len = 0;

   while (*s != '\0') { len++; s++; }

   return len;

}


void strCopy(char *dest, char *src) {

   while (*src != '\0') { *dest = *src; dest++; src++; }

   *dest = '\0';

}


void strConcat(char *s1, char *s2) {

   while (*s1 != '\0') s1++;

   while (*s2 != '\0') { *s1 = *s2; s1++; s2++; }
```

```
    *s1 = '\0';
}

int strCompare(char *s1, char *s2) {
    while (*s1 && *s2) {
        if (*s1 != *s2) return *s1 - *s2;
        s1++; s2++;
    }
    return *s1 - *s2;
}

int main() {
    char a[50] = "Hello", b[50] = "World", c[50];
    cout << "Length of a = " << strLength(a) << endl;
    strCopy(c, a); cout << "Copy of a = " << c << endl;
    strConcat(a, b); cout << "Concatenation = " << a << endl;
    cout << "Comparison result = " << strCompare("abc","abd") << endl;
    return 0;
}
```

Logic: Manual string operations using pointers. **Example Output:**

Code

Length of a = 5

Copy of a = Hello

Concatenation = HelloWorld

Comparison result = -1

27. Exceptions (basic try–catch)

Program:

cpp

```
#include <iostream>

using namespace std;


int main() {

    try {

        int a, b;

        cout << "Enter a and b: ";

        cin >> a >> b;

        if (b == 0) throw runtime_error("Division by zero!");

        cout << "Result = " << a / b << endl;

    } catch (exception &e) {

        cout << "Error: " << e.what() << endl;

    }

    return 0;

}
```

Logic: Handles division by zero using try–catch. **Example:** Input: 10 0 → Output: Error: Division by zero!

28. Custom Exception Class

Program:

cpp

```
#include <iostream>

#include <stdexcept>

using namespace std;
```

```
class MyException : public exception {
public:
    const char* what() const noexcept override {
        return "Custom Exception Occurred!";
    }
};

int main() {
    try {
        throw MyException();
    } catch (exception &e) {
        cout << e.what() << endl;
    }
    return 0;
}
```

Logic: User-defined exception class. **Example Output:**

Code

Custom Exception Occurred!

30. File Operations

Program:

cpp

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
```

```
    ofstream fout("data.txt");
    fout << "Hello File" << endl;
    fout.close();

    ifstream fin("data.txt");
    string line;
    while (getline(fin, line)) {
        cout << line << endl;
    }
    fin.close();
    return 0;
}
```

Logic: Write to file, then read back. **Example Output:**

Code

Hello File

31. Templates (Function Template)

Program:

cpp

```
#include <iostream>
using namespace std;

template <class T>
T add(T a, T b) { return a + b; }

int main() {
    cout << add<int>(5, 6) << endl;
```

```
    cout << add<double>(2.5, 3.5) << endl;
    return 0;
}
```

Logic: Generic function works